

Architectural Reflection: Lab 1 — Adapter Pattern

Course: Integrative Programming and Technologies (IPT)

Module: Structural Design Patterns in Web Application Development

Topic: Incompatible File Writers (CSV / JSON / Text / XML)

1. Spotting the Smell: Format Branching and OCP Violation

In the starter codebase (`without_web.php`), the `ReportExporterWithoutAdapter` class suffers from severe architectural coupling. To export attendance time-logs across multiple formats, the class employs conditional branching:

```
if ($this->format === 'csv') { ... }

elseif ($this->format === 'json') { ... }

elseif ($this->format === 'text') { ... }
```

This design exhibits several critical anti-patterns common in legacy web applications:

1. **Open-Closed Principle (OCP) Violation:** Whenever a client requests an additional format (such as XML, XLSX, or PDF), developers must directly modify the core export class, risking regression in existing, stable formats.
 2. **Vendor / Subsystem Leakage:** Low-level I/O mechanics (`fputcsv`, `json_encode`, raw line-by-line `fwrite` file pointers) are intermingled with high-level reporting logic. The exporter must know every library's specific signature and idiosyncrasies.
 3. **High Coupling & Shotgun Surgery:** If an underlying library's signature changes (e.g., streaming vs. bulk memory), every caller branching on that format must be updated.
-

2. Architectural Fix: The GoF Adapter Pattern

The refactored implementation (`with_pattern.php`) introduces the **Adapter Pattern** to convert the heterogeneous interfaces of legacy libraries (`CsvLibrary`, `JsonLibrary`, `TextLibrary`, and `XmlLibrary`) into a single, cohesive target contract:

```
interface FileWriter {
```

```
public function write(string $filename, array $rows): bool;

}
```

Each format is encapsulated within its own dedicated adapter class (`CsvFileAdapter`, `JsonFileAdapter`, `TextFileAdapter`, `XmlWriterAdapter`). These adapters implement `FileWriter` and hold a reference to their respective adaptees via constructor property promotion:

- **Client Decoupling:** `ReportExporter` now depends strictly on the `FileWriter` abstraction (`ReportExporter(FileWriter $writer)`), satisfying the **Dependency Inversion Principle (DIP)**.
 - **Single Responsibility Principle (SRP):** Format-specific transformations (e.g., text pipe-delimited serialization or XML DOM node building) live strictly within their respective adapter classes.
-

3. Academic Grounding (Gamma et al., 1994)

"Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces."

— Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (*Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, p. 139)

Buschmann et al. (1996) categorize the Adapter as an **Integration Pattern**. In modern software engineering, legacy components, third-party vendor SDKs, and evolving API versions cannot always be rewritten. The Adapter provides a translation layer without modifying existing, tested vendor code.

4. Extensibility Demonstration & Practical Web Applications

To prove extensibility, a new format—**XML file export**—was added to the system by introducing a single new class, `XmlWriterAdapter`, wrapping an `XmlLibrary`. The core `ReportExporter` class and existing adapters required **zero line modifications** ($O(1)$ change), proving full conformance to the Open-Closed Principle.

Real-World Web Applications in PHP Frameworks

- **File Storage (Laravel Flysystem):** `Storage::disk('s3')` and `Storage::disk('local')` implement `FilesystemAdapter`, translating unified `put()` / `get()` calls to AWS S3 SDK or PHP filesystem calls.
- **Cache Abstraction (PSR-16 / PSR-6):** Unified `CacheInterface::get($key)` adapted to Redis, Memcached, APCu, or database stores.
- **Payment Gateways:** Standardizing e-commerce checkouts across Stripe, PayPal, and PayMongo without modifying the order checkout service.